Floating point numbers

CSE 101 Section 03
Introduction to Algorithms and Computing
Fall 2022
© Royal Military College of Canada

Floating-Point Number Anatomy

$$-3.154 * 10^{5}$$

The **sign** (negative here)
The **mantissa** (= `3.154` here)
The **exponent** (= `5` here)

Floating-Point in Binary

In decimal the number `56.482` translates as

$5 * 10^1$	50
$6 * 10^0$	6
$4 * 10^{-1}$	4/10
$8 * 10^{-2}$	8/100
$2 * 10^{-3}$	2/1000

In binary it is the same process however we use powers of `2` instead.

So in binary the number `101.101` translates as

$1 * 2^2$	4
$0 * 2^1$	0
$1 * 2^0$	1
$1 * 2^{-1}$	1/2
$0 * 2^{-2}$	0
$1 * 2^{-3}$	1/8

Floating-Point in Binary

101.101 (binary)

$1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$

$4 + 0 + 1 + 1/2 + 0 + 1/8$

$4 + 1 + 0.5 + 0.125$

5.625 (decimal)

The binary point

- So far we have represented our binary fractions with the use of a binary point.
- This is fine when we are working with things normally but within a computer this is not feasible as it can only work with `0`'s and `1`'s.
  - We get around this by agreeing where the binary point should be.

- So, for instance, if we are working with `8` bit numbers, it may be agreed that the binary point will be placed between the `4`th and `5`th bits.

`01101001` is then assumed to actually represent `0110.1001`

Converting a decimal fraction to binary

- The easiest approach is a method where we repeatedly multiply the fraction by `2` and recording whether the digit to the left of the decimal point is a `0` or `1` (ie, if the result is greater than `1`), then discarding the `1` if it is. Once you are done you read the value from top to bottom.

`0.16`

0.32	0
0.64	0
1.28	1
0.56	0
1.12	1
0.24	0
0.48	0
0.96	0

`0.00101000`

Floating Point

- What we have looked at previously is what is called fixed point binary fractions, convenient but….
- As soon as the number we want to represent is very large or very small we find that we need a very large number of bits to represent them.

- To get around this we use a method of representing numbers called **floating point.**
- Floating point is quite similar to scientific notation as a means of representing numbers.

Back to this: Floating-Point Number Anatomy

$$-3.154 * 10^{5}$$

The **sign** (negative here)
The **mantissa** (= `3.154` here)
The **exponent** (= `5` here)

Now the same in Binary

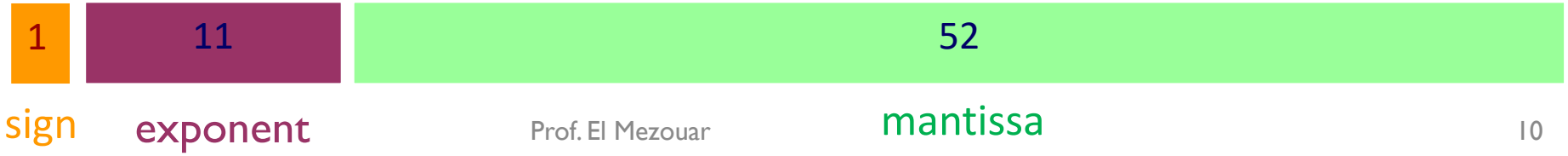
- We may do the same in binary and this forms the foundation of our floating point number.

- Matlab uses what is called the IEEE 754 Standard, which tells us exactly how to represent floating point numbers.
  - The standard specifies the number of bits used for each section (exponent, mantissa and sign).
  - and the order in which they are represented

IEEE 754: Number of bits and Order

- Single precision uses `32` bits and has the following layout:
  - **1 bit** for the sign of the number. `0` means positive and `1` means negative.
  - **8 bits** for the exponent.
  - **23 bits** for the mantissa.

- Double precision uses `64` bits and has the following layout.
  - **1 bit** for the sign of the number. `0` means positive and `1` means negative.
  - **11 bits** for the exponent.
  - **52 bits** for the mantissa.



Sign bit

- This is the first bit (left most bit) in the floating point number and it is pretty straightforward.
- If your number is positive, make this bit a `0`. If your number is negative then make it a `1`.

The Exponent

- The exponent gets a little interesting.
- Remember that the exponent can be positive (to represent large numbers) or negative (to represent small numbers, i.e. fractions).

Assume single precision (i.e. 16 bits total and 8 bits for the exponent):

- To allow for negative exponents, do the following:
  - Take exponent and add 127 to it
  - Convert to binary and store in the exponent section.

- Examples:
  - We want our exponent to be `5: 5 + 127` is `132` so our exponent becomes `10000100`
  - We want our exponent to be `-7. -7 + 127` is `120` so our exponent becomes `01111000`

The Mantissa

- In scientific notation, remember that we move the point so that there is only a single (non zero) digit to the left of it (`0.32` becomes $3.2*10^{-1}$).

- We do the same with binary…
- E.g. if our number to store was `111.00101101`, then in scientific notation it would be `1.1100101101` with an exponent of `2`

Converting to Floating point Binary

Converting a number to floating point involves the following steps:

- **Set the sign bit -** if the number is positive, set the sign bit to 0. If the number is negative, set it to 1.
- **Divide your number into two sections -** the whole number part and the fraction part.
- **Convert to binary -** convert the two numbers into binary then join them together with a binary point.
- **Work out the exponent -** This is done by working out how many spaces the binary point needs to be moved so that it is just after the first **1** in the result. If you move the binary point to the left then this number is positive. If you move it to the right then the number is negative. Add 127 to this number then convert to binary.
- **Format the mantissa -** This is done by dropping the first **1** in the number and recording the next 23 bits.

Example: `-16.1`

1	Negative so sign bit is 1
10000	Convert whole number section to binary
0.00011001100110011001100110	Convert fraction section to binary
10000.00011001100110011001100	Join the two together
4	How many spaces to move binary point
10000011	Add 127 to get Exponent
10000011	Convert Exponent to binary
1.00000001100110011001100	Adjust the mantissa
00000001100110011001100	Remove the leading 1

Result in Binary : **1 10000011 00000001100110011001100**

Next…

Rounding errors in FP numbers